



PROMPT ENGINEERING

Week 2 – Day 1 Assignment



JUNE 16, 2026

ANITHA PRAJISH

Contents

Scenario Name — Search Functionality.....2

 Weak Prompt2

 What I Feel is Missing2

 Rewritten Prompt2

 Observations4

Scenario Name — Shopping Cart.....5

 Weak Prompt5

 What I Feel is Missing5

 Rewritten Prompt5

 Observations7

Scenario Name — Dashboard / Report Generation8

 Weak Prompt8

 What I Feel is Missing8

 Rewritten Prompt8

 Observations 11

Scenario Name — Search Functionality

Weak Prompt

Test the search feature.

What I Feel is Missing

- Overview of the feature's functionality
- User requirements for the feature
- Expected system behaviour
- Types of testing being conducted
- User roles and access permissions
- Search input parameters (text type, length limits, case sensitivity, supported languages)
- Output structure (sorting, filtering, pagination)
- Edge case handling (invalid, incorrect, or missing data; network failures)
- Application performance (response time, capacity handling)
- Backend dependencies (databases, APIs)
- Supported platforms (web, mobile, browser compatibility)
- Test data requirements (sample datasets)
- Security requirements and constraints

Rewritten Prompt

INSTRUCTION:

[MANDATORY] Create detailed test scenarios covering functional and non-functional requirements.

[MANDATORY] The application is an eCommerce platform for ordering plumbing parts, supporting both B2B and B2C users.

[HIGH PRIORITY] Search should support alphanumeric input, be case-insensitive, and support English (US), including partial matches and typo tolerance.

[HIGH PRIORITY] Results should be sorted by relevance, price, and popularity, support filtering (category, price range, ratings), and pagination (10/20/50 items per page).

[HIGH PRIORITY] Search should handle invalid, incorrect, and empty inputs, including no-result scenarios.

[CRITICAL] Ensure robust error handling for no results, network failures, and latency issues.

[MANDATORY] Validate performance (response time <2 sec, high concurrency), backend reliability (database, APIs, caching), and security (input validation, rate limiting, bot protection).

[HIGH PRIORITY] Define user roles and access (Guest, Logged-in, Admin) and validate role-based behaviour.

[HIGH PRIORITY] Ensure compatibility across Web (browsers) and Mobile (iOS/Android).

[MEDIUM] Include validation for search relevance, duplicate results, auto-suggestions, and usability aspects.

[MEDIUM] Use appropriate test data sets, including large product catalogues.

CONTEXT:

Act as a Senior Test Engineer to generate test cases. The application is a plumbing-focused eCommerce platform supporting both B2B and B2C users, where customers search for products such as pipes, fittings, valves, and accessories using keywords, filters, and categories. The search system supports auto-suggestions, spell correction, synonym matching, and relevance-based ranking to ensure accurate and efficient product discovery.

EXAMPLE:

Input: "PVC pipe 2 inch"

Expected Output:

- Relevant plumbing products are displayed (e.g., 2-inch PVC pipes, fittings)
- Results are sorted by relevance
- Response time is under 2 seconds
- No unrelated items (e.g., non-plumbing products) are shown

PERSONA:

Act as a Senior Test Lead specializing in eCommerce systems, responsible for reviewing test cases, ensuring complete coverage, identifying gaps, and validating all functional and non-functional scenarios.

OUTPUT:

Generate test cases in a structured format compatible with Azure DevOps (ADO) Test Plans for direct import/upload.

Each test case must include:

- Title (Test Case Name)

- Test Suite / Area Path
- Priority (Mandatory / Critical / High / Medium)
- Test Type (Functional / Performance / Security / Usability / Edge Case)
- Preconditions
- Test Steps (clearly numbered)
- Test Data
- Expected Result

Ensure each test case is independent, clear, reusable, and suitable for automation.

TONE:

Use a professional, precise, and detail-oriented tone suitable for enterprise QA documentation, ensuring clarity, completeness, and actionable outputs.

Observations

Write your first observation on what improved in the output

- Clearly defined scope instead of vague intent
- Structured instructions with priorities
- Covers both functional and non-functional aspects
- Explicit search behaviour requirements
- Detailed output expectations
- Includes edge cases and error handling scenarios
- Defines performance benchmarks
- Specifies backend dependencies
- Includes security considerations
- Covers user roles and access control validation
- Ensures cross-platform testing coverage
- Adds usability aspects
- Provides clear example for guidance
- Defines persona for expert-level validation
- Specifies structured output format
- Improves clarity, completeness, and consistency

Scenario Name — Shopping Cart

Weak Prompt

Test the cart functionality.

What I Feel is Missing

- Specify the application type (web / mobile / e-commerce).
- Mention user types (guest and logged-in users).
- Include core actions (add, update, remove items).
- Cover edge cases (out-of-stock, max quantity, invalid input).
- Validate price calculation (total, tax, discount, shipping).
- Check cart persistence (refresh, login, logout).
- Verify UI behaviour (messages, button states, updates).
- Consider performance (response time, loading speed).
- Include integration points (inventory, payment, offers).
- Define clear acceptance criteria (pass/fail conditions).

Rewritten Prompt

INSTRUCTION:

[MANDATORY] Create detailed test scenarios covering functional and non-functional requirements of the cart functionality.

[MANDATORY] The application is an eCommerce platform for ordering plumbing parts, supporting both B2B and B2C users.

[MANDATORY] Focus on cart functionality only; exclude payment and order placement unless directly related to cart validation.

[MANDATORY] Validate core cart operations including add to cart, update quantity, remove items, and clear cart.

[CRITICAL] Ensure accurate price calculation including product price, taxes, discounts, shipping charges, and total amount.

[CRITICAL] Verify stock validation for plumbing items (in-stock, out-of-stock, low inventory) and prevent invalid additions.

[CRITICAL] Ensure cart persistence across sessions including refresh, login, logout, and device switching.

[HIGH PRIORITY] Validate handling of duplicate items, quantity limits, and bulk additions for B2B users.

[HIGH PRIORITY] Verify application of promo codes, discounts, and special offers, including invalid and expired cases.

[HIGH PRIORITY] Ensure proper handling of invalid inputs, out-of-stock products, and edge cases like exceeding maximum quantity.

[CRITICAL] Ensure robust error handling for failures such as inventory mismatch, pricing errors, session timeout, and API/network issues.

[MANDATORY] Validate performance (cart operations response time <2 sec, handling multiple items), backend reliability (APIs, database, caching), and security (input validation, session handling).

[HIGH PRIORITY] Define user roles (Guest, Logged-in, B2B Customer, Admin) and validate role-based cart behaviour.

[HIGH PRIORITY] Ensure compatibility across Web (browsers) and Mobile (iOS/Android) platforms.

[MEDIUM] Validate UI/UX aspects such as real-time updates, error/success messages, and usability of cart interactions.

[MEDIUM] Use appropriate test data sets including bulk plumbing products, varied pricing, and large cart volumes.

CONTEXT:

Act as a Senior Test Engineer to generate test cases. The application is a plumbing-focused eCommerce platform supporting both B2B and B2C users, where customers manage cart functionality for products such as pipes, fittings, valves, and accessories. The cart system allows users to add, update, and remove items, apply discounts, and proceed to checkout. It is integrated with inventory, pricing, and promotion services to ensure real-time stock validation, accurate pricing, and seamless user experience. The system supports cart persistence, role-based behaviour (guest and logged-in users), and ensures efficient and reliable cart operations across web and mobile platforms.

EXAMPLE:

Input: Add 2 units of "PVC pipe 2 inch" to the cart

Expected Output:

- The selected product is successfully added to the cart
- Cart displays correct product name, quantity, and price
- Total price is updated accurately based on quantity
- Stock availability is validated before adding
- User receives confirmation message (e.g., "Item added to cart")
- Cart updates in real time without page errors

PERSONA:

Act as a Senior Test Lead specializing in eCommerce systems, responsible for reviewing cart-related test cases, ensuring complete coverage of functional and non-functional scenarios, identifying gaps in cart operations, and validating critical workflows such as item management, pricing accuracy, stock validation, and cart persistence across user roles (B2B and B2C).

OUTPUT:

Generate test cases in a structured format compatible with Azure DevOps (ADO) Test Plans for direct import/upload.

Each test case must include:

- Title (Test Case Name)
- Test Suite / Area Path
- Priority (Mandatory / Critical / High / Medium)
- Test Type (Functional / Performance / Security / Usability / Edge Case)
- Preconditions
- Test Steps (clearly numbered)
- Test Data
- Expected Result

Ensure each test case is independent, clear, reusable, and suitable for automation.

TONE:

Use a professional, precise, and detail-oriented tone suitable for enterprise QA documentation, ensuring clarity, completeness, and actionable outputs.

Observations

Write your first observation on what improved in the output

- Clearly defined scope instead of vague intent
- Structured instructions with priorities
- Covers both functional and non-functional aspects
- Explicit cart behaviour requirements
- Detailed expectations for pricing, stock validation, and user interactions
- Includes edge cases and error handling scenarios
- Defines performance benchmarks
- Specifies backend dependencies
- Includes security considerations
- Covers user roles and access control validation
- Ensures cross-platform testing coverage
- Adds usability aspects
- Provides clear example for guidance
- Defines persona for expert-level validation
- Specifies structured output format
- Improves clarity, completeness, and consistency
- Transforms a vague request into a detailed, execution-ready QA prompt

Scenario Name — Dashboard / Report Generation

Weak Prompt

Test the reports section.

What I Feel is Missing

- Specify the application type (web / mobile / e-commerce / enterprise reporting dashboard).
- Identify user roles (admin, analyst, manager, guest with limited access).
- Validate report generation actions (create, view, export, schedule, delete reports).
- Verify filters and parameters (date range, categories, custom inputs, invalid filters).
- Check data accuracy (report data should match database/source records).
- Cover edge cases (no data available, large datasets, invalid date ranges, failed generation).
- Validate calculations (totals, averages, trends, percentages, aggregations).
- Verify export functionality (PDF, Excel, CSV formats and file correctness).
- Check report refresh and caching (real-time updates vs cached data behavior).
- Validate UI behaviour (loading indicators, empty state messages, error notifications).
- Verify sorting, pagination, and search within reports.
- Check access control (permissions for viewing/downloading sensitive reports).
- Test performance (report generation time, large data load handling, timeout scenarios).
- Validate integration points (database, APIs, analytics tools, third-party BI systems).
- Verify scheduling and notifications (automated reports, email delivery, failure alerts).
- Check localization
- Define clear acceptance criteria (correct data, proper formatting, successful export, no errors).

Rewritten Prompt

INSTRUCTION:

[MANDATORY] Create detailed test scenarios covering functional and non-functional requirements of the report functionality.

[MANDATORY] The application is a plumbing-focused eCommerce platform supporting both B2B and B2C users, where senior management generates reports for business insights.

[MANDATORY] Focus only on the report module; exclude cart, payment, and order placement unless required for validating report data accuracy.

[MANDATORY] Validate end-to-end report functionality including generation, filtering, summary view, drill-down, export (PDF/Excel/CSV), scheduling, and refresh behaviour.

[CRITICAL] Ensure data accuracy, consistency, and integrity across all reports (sales, revenue, taxes, discounts, inventory, high-demand products) by validating against backend systems (orders, inventory, pricing, customer data).

[CRITICAL] Validate inventory/stock reporting (in-stock, low stock, out-of-stock, replenishment) and ensure drill-down data aligns with summarized values.

[CRITICAL] Validate export functionality for correctness, formatting, completeness, encoding, and consistency with on-screen data.

[CRITICAL] Ensure robust error handling and edge case coverage including invalid filters, no data scenarios, large datasets, report failures, timeouts, and API/network issues.

[MANDATORY] Validate performance, scalability, and reliability including response time (<3 sec), large dataset handling, pagination, sorting, search, caching, and backend stability (APIs, database).

[MANDATORY] Validate security and access control including role-based permissions, data visibility restrictions, and prevention of unauthorized access to sensitive reports.

[HIGH PRIORITY] Validate different report types (sales, inventory, high-demand products, customer trends) and filter functionality (date ranges, product categories, customer types, combinations, invalid inputs).

[HIGH PRIORITY] Validate report scheduling (frequency, accuracy of generated data, notification delivery) and refresh behaviour (real-time vs cached data, filter retention).

[HIGH PRIORITY] Validate user roles (Senior Management, Manager, Analyst, Admin, Warehouse Manager), cross-platform compatibility (Web/Mobile), and role-based report behaviour.

[MEDIUM] Validate UI/UX (charts, tables, loading indicators, empty states, readability) and usability features.

[MEDIUM] Validate audit logging (report generation, export, access), time zone consistency, and use of realistic high-volume test data (sales, inventory, B2B transactions, seasonal demand).

[MANDATORY] Define clear acceptance criteria including data accuracy, successful report generation, correct filters, accurate exports, and error-free execution.

CONTEXT

Act as a Senior Test Engineer to generate test cases. The application is a plumbing-focused eCommerce platform supporting both B2B and B2C users, where senior management and business users access the reports module for insights into operations. The reports system enables users to generate, view, filter, and export reports related to sales performance, inventory/stock levels, high-demand plumbing products (pipes, fittings, valves, accessories), and customer purchasing trends. It is integrated with order management, inventory, pricing, and analytics services to ensure accurate data representation, real-time updates, and reliable reporting. The system supports role-based access (senior management, managers, analysts), report scheduling, and handles large datasets efficiently, ensuring consistent performance and usability across web and mobile platforms.

Example:

Input: Generate a sales report for "PVC pipes" for the date range Jan 1 – Jan 31

Expected Output:

- The report is successfully generated based on the selected filters
- Report displays correct product category, date range, and sales data
- Total sales, quantity sold, and revenue are calculated accurately
- Data matches backend order and transaction records
- High-demand items within the category are correctly highlighted
- User receives confirmation or loading indicator during report generation
- Report is displayed without errors and supports export (PDF/Excel)
- Data loads within acceptable response time without performance issues

Persona:

Act as a Senior Test Lead in eCommerce systems, responsible for reviewing Reports module test cases, ensuring comprehensive coverage of functional and non-functional scenarios, identifying workflow gaps, and validating key functionalities such as data accuracy, report generation, filtering, exports, and role-based access. Emphasize reliable data for informed decision-making and conduct risk validation to identify data inconsistencies, performance issues, and security risks impacting senior management operations in a plumbing-focused B2B and B2C platform.

OUTPUT:

Generate test cases in a structured format compatible with Azure DevOps (ADO) Test Plans for direct import/upload.

Each test case must include:

- Title (Test Case Name)
- Test Suite / Area Path

- Priority (Mandatory / Critical / High / Medium)
- Test Type (Functional / Performance / Security / Usability / Edge Case)
- Preconditions
- Test Steps (clearly numbered)
- Test Data
- Expected Result

Ensure each test case is independent, clear, reusable, and suitable for automation.

TONE:

Use a professional, precise, and detail-oriented tone suitable for enterprise QA documentation, ensuring clarity, completeness, and actionable outputs.

Observations

Write your first observation on what improved in the output

- Clearly defined scope instead of a generic “reports section” request
- Structured instructions with prioritization for better clarity and execution
- Covers both functional and non-functional aspects
- Explicit definition of report operations
- Strong focus on data accuracy, consistency, and integrity with backend validation
- Detailed validation of inventory and stock reporting
- Includes comprehensive export validation
- Covers edge cases and robust error handling
- Defines performance benchmarks and scalability expectations
- Specifies backend dependencies
- Includes strong security and access control validation
- Covers multiple report types
- Incorporates filter validation with combinations and invalid input scenarios
- Includes scheduling and refresh behaviour
- Covers user roles and permissions
- Ensures cross-platform compatibility
- Adds UI/UX validation
- Includes audit logging and compliance validation for traceability
- Uses realistic, high-volume domain-specific test data
- Accounts for time zone consistency in reporting
- Provides clear acceptance criteria for validation success
- Includes a strong business context explaining system behaviour and integrations
- Provides a realistic example input/output for better understanding
- Defines a clear expert persona with emphasis on decision-making and risk validation
- Specifies structured output format

- Ensures professional tone suitable for enterprise QA documentation
- Transforms a vague request into a comprehensive, execution-ready, and automation-friendly QA prompt